TEK4090

Introduction to Modern Control

Lecture 3: Optimal Control

Mathias Hudoba de Badyn

September 25, 2025

Plan for Rest of the Course

- Shift lectures by one

Plan for Rest of the Course

- Shift lectures by one
- Either record the last lecture, or find an alternate day

Plan for Rest of the Course

- Shift lectures by one
- Either record the last lecture, or find an alternate day
- Mario will record the Matlab tutorial

Plan for Rest of the Course

- Shift lectures by one
- Either record the last lecture, or find an alternate day
- Mario will record the Matlab tutorial
- Homework 1 grades: Monday; Homework 2: out after next lecture

Master Thesis Topics + Final Project Topics

- Master's thesis topics are available on the Cybernetics web page:
  www.mn.uio.no/its/studier/masteroppgaver/masters_thesis_CAS/

Master Thesis Topics + Final Project Topics

- Master's thesis topics are available on the Cybernetics web page: `www.mn.uio.no/its/studier/masteroppgaver/masters_thesis_CAS/`
- Homework 2 will include a portion on planning your final project

Master Thesis Topics + Final Project Topics

- Master's thesis topics are available on the Cybernetics web page: `www.mn.uio.no/its/studier/masteroppgaver/masters_thesis_CAS/`
- Homework 2 will include a portion on planning your final project
- As part of (or shortly after) Homework 2, you should set up a zoom/office meeting with me or Mario ($\approx$ 15–30 mins) to go over your project plan

Hydroponics

- ‘Digital Twins’ of plants

Hydroponics

- 'Digital Twins' of plants
- Map pictures via photogrammetry to a model of the plant growth

Hydroponics

- 'Digital Twins' of plants
- Map pictures via photogrammetry to a model of the plant growth
- Optimize the light, nutrient mix, oxygenation, etc. to make the 'best' vegetable

Hydroponics

- 'Digital Twins' of plants
- Map pictures via photogrammetry to a model of the plant growth
- Optimize the light, nutrient mix, oxygenation, etc. to make the 'best' vegetable
- No funny business (eg., the English violet, an invasive plant)

Hydroponics

- 'Digital Twins' of plants
- Map pictures via photogrammetry to a model of the plant growth
- Optimize the light, nutrient mix, oxygenation, etc. to make the 'best' vegetable
- No funny business (eg., the English violet, an invasive plant)
- I am testing the equipment in the basement with Carolina reapers

Drone Swarms

- Real-time optimization

Drone Swarms

- Real-time optimization
- We have (or, will have) three different drone swarm platforms

Drone Swarms

- Real-time optimization
- We have (or, will have) three different drone swarm platforms
- Both 'high-level' (eg., simulink blocks) or 'low-level' (eg., C/C++)

Smart Cities

- Possibility to work with Lillestrøm Kommune

Smart Cities

- Possibility to work with Lillestrøm Kommune
- Or, some data from buildings in Switzerland

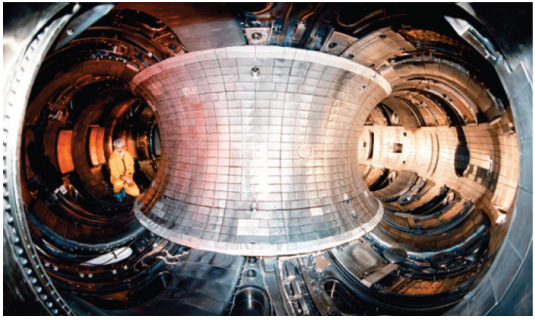
Smart Cities

- Possibility to work with Lillestrøm Kommune
- Or, some data from buildings in Switzerland
- Or, with data from a distribution grid in Switzerland

Smart Cities

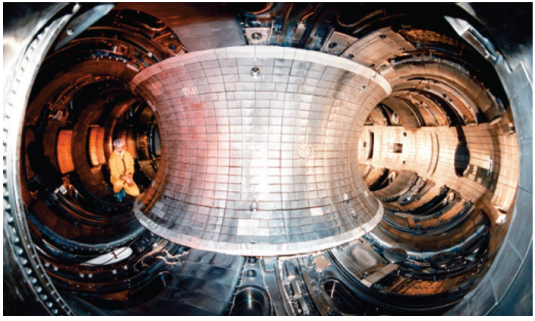
- Possibility to work with Lillestrøm Kommune
- Or, some data from buildings in Switzerland
- Or, with data from a distribution grid in Switzerland
- Or, a million other things

Fusion Reactor Control



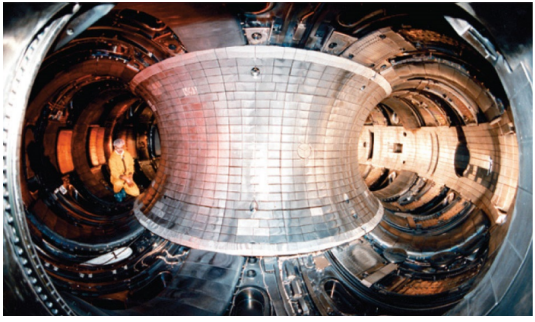
- Fusion reactors can't do feedback control easily

Fusion Reactor Control



- Fusion reactors can't do feedback control easily
- Need to design control 'iteratively' over multiple firings

Fusion Reactor Control

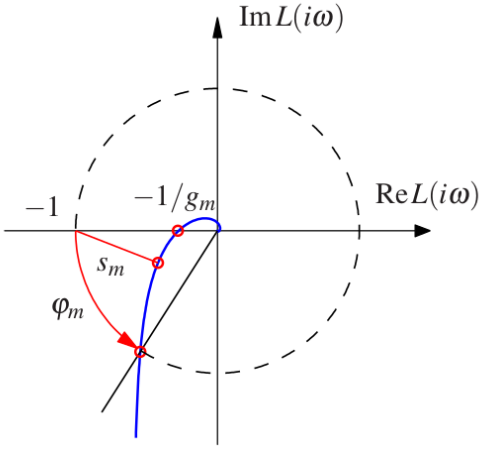


- Fusion reactors can't do feedback control easily
- Need to design control 'iteratively' over multiple firings
- If you know plasma physics, this would be an "easy" publication

Reading Assignment

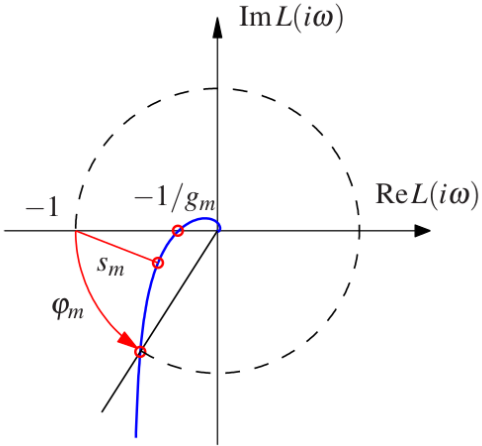
1. **Lewis, Syrmos, Vraibie:** Ch2,3 & 6. Warning: typos galore

Recap from Last Time



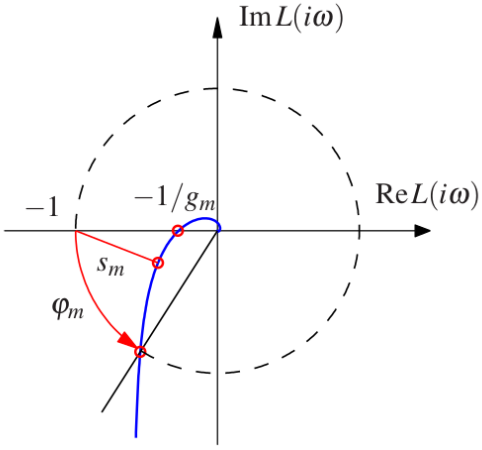
- Nyquist stability and Bode Plots

Recap from Last Time



- Nyquist stability and Bode Plots
- Stability margins

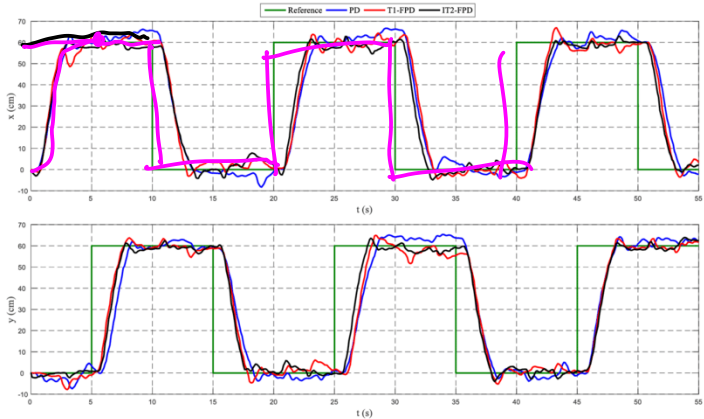
Recap from Last Time



- Nyquist stability and Bode Plots
- Stability margins
- Introduction to Linear Systems

Motivation for Stabilization

System is stable $\implies x(t) \to r(t)$



Stabilization by State Feedback

$\dot{x} = Ax + Bu$

$y = Cx + Du$

$u = -Kx + k_f r$ — feedback, reference

with measurement $u = -Ky + k_f r$

$\dot{x} = (A - BK)x + Bk_f r$ } want this to be stable

$\bar{A}$

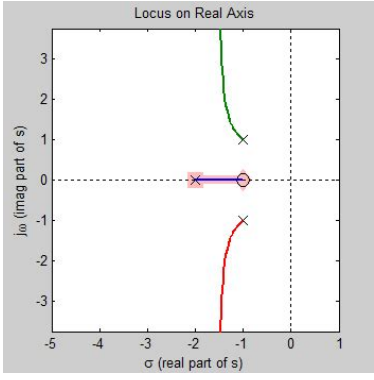
{ $\det(\lambda I - \bar{A}) = 0$ } -> solutions to have negative real part

Stabilization by State Feedback

Motivation for Optimal Control

But how do we even pick the gains properly?

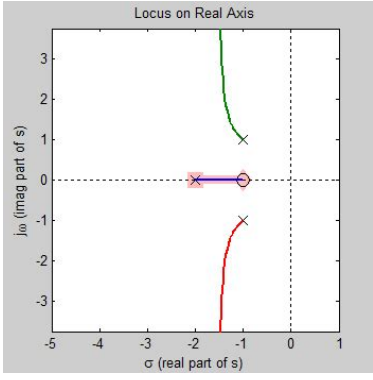
- Potentially by staring at the Bode plots and making sure the frequency response fits some criteria



Motivation for Optimal Control

But how do we even pick the gains properly?

- Potentially by staring at the Bode plots and making sure the frequency response fits some criteria
- **Root Locus:** plot the 'trajectory' of the poles in the complex plane as a function of k_i's



Motivation for Optimal Control

Control is the math behind automation. Why don't we automate picking the gains?

Motivation for Optimal Control

Control is the math behind automation. Why don't we automate picking the gains?

Just pick the best one

Motivation for Optimization

Ok, but, how?

- General recipe for optimal control:

Motivation for Optimization

Ok, but, how?

- General recipe for optimal control:
- Define a performance metric for your system

Motivation for Optimization

Ok, but, how?

- General recipe for optimal control:
- Define a performance metric for your system
- Do some calculus to optimize the metric

Motivation for Optimization

Ok, but, how?

- General recipe for optimal control:
- Define a performance metric for your system
- Do some calculus to optimize the metric
- Give up and do machine learning?

basics of (un)constrained optimization

Unconstrained optimization

- **unconstrained optimization**:

$$\underset{x \in \mathbb{R}^n}{\text{minimize}}\, f(x) \tag{1}$$

where $f : \mathbb{R}^n \to \mathbb{R}$ is a continuously differentiable *objective* function

Unconstrained optimization

- **unconstrained optimization**:

$$\underset{x \in \mathbb{R}^n}{\text{minimize}}\, f(x) \tag{1}$$

  where $f : \mathbb{R}^n \to \mathbb{R}$ is a continuously differentiable *objective* function

- $x^\star \in \mathbb{R}^n$ is said to be a **minimizer** and $f(x^\star)$ is a **minimum** if

$$f(x^\star) \leq f(x) \qquad \text{for all } x \in \mathbb{R}^n$$

Unconstrained optimization

- **unconstrained optimization**:

$$\underset{x \in \mathbb{R}^n}{\text{minimize}}\, f(x) \tag{1}$$

  where $f : \mathbb{R}^n \to \mathbb{R}$ is a continuously differentiable *objective* function

- $x^\star \in \mathbb{R}^n$ is said to be a **minimizer** and $f(x^\star)$ is a **minimum** if

$$f(x^\star) \leq f(x) \qquad \text{for all } x \in \mathbb{R}^n$$

- a minimizer and minimum are said to be **strict** if

$$f(x^\star) < f(x) \qquad \text{for all } x \in \mathbb{R}^n \setminus \{x^\star\}.$$

Note: all concepts are defined globally, but there are also local versions

Unconstrained optimization

- **unconstrained optimization**:

$$\underset{x \in \mathbb{R}^n}{\text{minimize}}\ f(x) \tag{1}$$

where $f : \mathbb{R}^n \to \mathbb{R}$ is a continuously differentiable *objective* function

- $x^\star \in \mathbb{R}^n$ is said to be a **minimizer** and $f(x^\star)$ is a **minimum** if

$$f(x^\star) \leq f(x) \qquad \text{for all } x \in \mathbb{R}^n$$

- a minimizer and minimum are said to be **strict** if

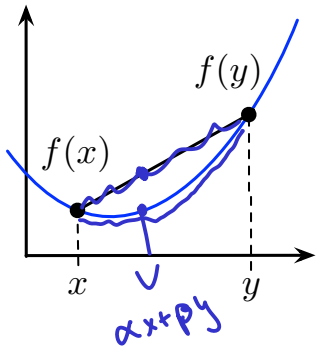
$$f(x^\star) < f(x) \qquad \text{for all } x \in \mathbb{R}^n \setminus \{x^\star\}.$$

Note: all concepts are defined globally, but there are also local versions

- on **gradients**: $\frac{\partial f}{\partial x} : \mathbb{R}^n \to \mathbb{R}^n$ is the *column vector* of partial derivatives

Convexity

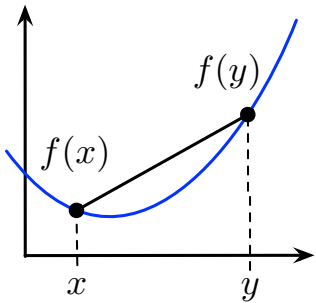
- f is said to be **convex** if "function is below line segment"



$$\forall x, y \in \mathbb{R}^n \text{ and } \forall \alpha, \beta \geq 0 \text{ with } \alpha + \beta = 1$$
$$f(\alpha x + \beta y) \leq \alpha f(x) + \beta f(y)$$

Convexity

- f is said to be **convex** if "function is below line segment"

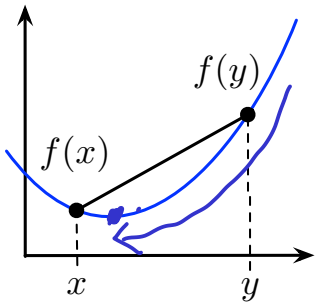


$$\forall x, y \in \mathbb{R}^n \text{ and } \forall \alpha, \beta \geq 0 \text{ with } \alpha + \beta = 1$$
$$f(\alpha x + \beta y) \leq \alpha f(x) + \beta f(y)$$

$\Rightarrow$ f is **strictly convex** if inequality is strict

Convexity

- f is said to be **convex** if "function is below line segment"



$$\forall x, y \in \mathbb{R}^n \text{ and } \forall \alpha, \beta \geq 0 \text{ with } \alpha + \beta = 1$$
$$f(\alpha x + \beta y) \leq \alpha f(x) + \beta f(y)$$

$\Rightarrow$ f is **strictly convex** if inequality is strict

$\Rightarrow$ f is **concave** if $-f$ is convex

Convexity

- **underestimator property**: "convex iff function above its tangents"

a cont. diff. function f is convex
if and only if for all $x, y \in \mathbb{R}^n$

$$f(y) \geq f(x) + (y - x)^\top \frac{\partial f(x)}{\partial x}$$

(strictly convex if inequality is strict for $x \neq y$)

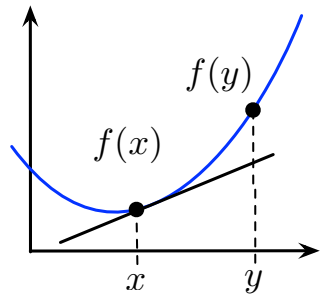
Convexity

- **underestimator property**: "convex iff function above its tangents"

a cont. diff. function f is convex
if and only if for all $x, y \in \mathbb{R}^n$

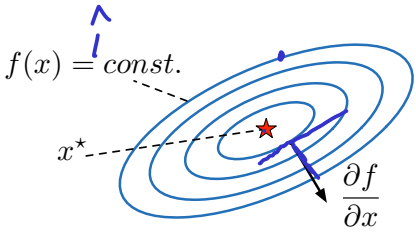
$$f(y) \geq f(x) + (y - x)^\top \frac{\partial f(x)}{\partial x}$$

(strictly convex if inequality is strict for $x \neq y$)

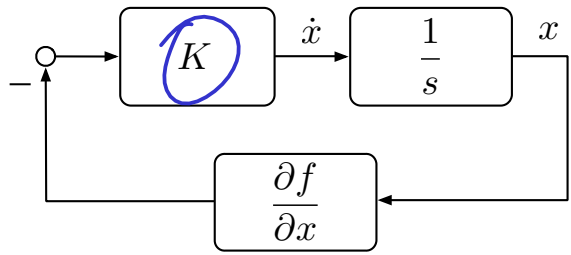


Unconstrained optimization & gradient flows

$$\dot{x} = -K \frac{\partial f(x)}{\partial x}$$



geometry of an unconstrained optimization problem



block-diagram of negative gradient flow with positive definite gain $K \in \mathbb{R}^{n \times n}$

Constrained optimization

- **linearly-constrained optimization:**

$$\begin{array}{ll} \text{minimize}_{x \in \mathbb{R}^n} & f(x) \\ \text{s.t.} & g(x) = \mathbf{0} \end{array} \tag{2}$$

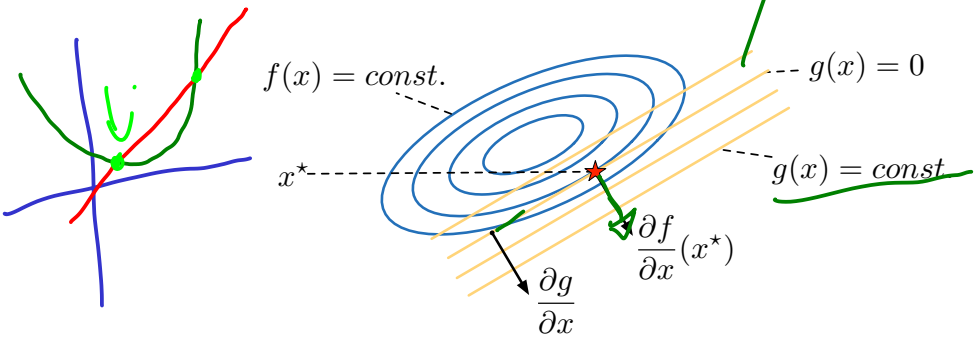
where f is continuously differentiable & $g(x) = Ax - b$ is **linear-affine**

Constrained optimization

- **linearly-constrained optimization:**

$$\begin{array}{ll} \text{minimize}_{x\in\mathbb{R}^n} & f(x) \\ \text{s.t.} & g(x) = \mathbf{0} \end{array} \tag{2}$$

where f is continuously differentiable & $g(x) = Ax - b$ is **linear-affine**

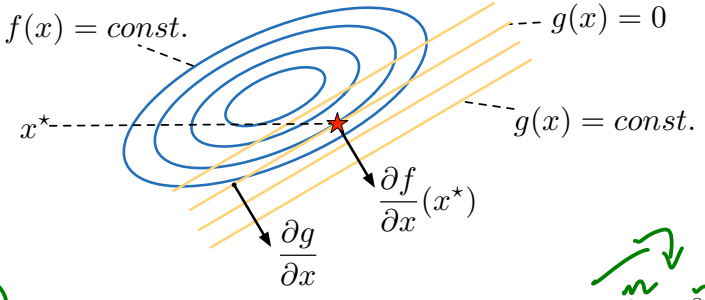


Constrained optimization

- **linearly-constrained optimization:**

$$\begin{array}{ll} \text{minimize}_{x\in\mathbb{R}^n} & f(x) \\ \text{s.t.} & g(x) = \mathbf{0} \end{array} \tag{2}$$

where f is continuously differentiable & $g(x) = Ax - b$ is **linear-affine**



$\Rightarrow$ minimizer $x^\star$ when $\{g(x) = \mathbf{0}\}$ **tangent** to $\{f(x) = const.\} \Leftrightarrow \frac{\partial f}{\partial x} \parallel \frac{\partial g}{\partial x}$

Constrained Optimization

- **linearly-constrained optimization:**

$$\begin{array}{ll} \text{minimize}_{x \in \mathbb{R}^n} & f(x) \\ \text{s.t.} & g(x) = \mathbf{0} \end{array} \tag{3}$$

where f is continuously differentiable & $g(x) = Ax - b$ is linear-affine

$\Rightarrow$ minimizer $x^\star$ satisfies **KKT conditions** (after Karush, Kuhn, Tucker):

KKT Conditions

(i) constraint equation: $\mathbf{0} = g(x) = Ax - b$

(ii) tangency condition: $\mathbf{0} = \dfrac{\partial f(x)}{\partial x} + \dfrac{\partial g(x)}{\partial x}^\top \lambda,$

where λ is Lagrange multiplier or *dual variable* (analogous: x is *primal variable*)

Constrained Optimization

- **linearly-constrained optimization:**

$$\begin{array}{ll} \text{minimize}_{x \in \mathbb{R}^n} & f(x) \\ \text{s.t.} & g(x) := Ax - b = \mathbf{0} \end{array} \tag{4}$$

KKT Conditions

$$(i) \text{ constraint equation: } \mathbf{0} = g(x) = Ax - b$$

$$(ii) \text{ tangency condition: } \mathbf{0} = \frac{\partial f(x)}{\partial x} + \frac{\partial g(x)}{\partial x}^{\top} \lambda$$

1. the KKT conditions are necessary conditions for optimality of $x^\star$

Constrained Optimization

- **linearly-constrained optimization:**

$$\begin{array}{ll} \text{minimize}_{x \in \mathbb{R}^n} & f(x) \\ \text{s.t.} & g(x) := Ax - b = \mathbf{0} \end{array} \tag{4}$$

KKT Conditions

$$(i) \text{ constraint equation: } \mathbf{0} = g(x) = Ax - b$$

$$(ii) \text{ tangency condition: } \mathbf{0} = \frac{\partial f(x)}{\partial x} + \frac{\partial g(x)}{\partial x}^{\top} \lambda$$

1. the KKT conditions are necessary conditions for optimality of $x^\star$
2. if f is convex, then the KKT conditions are sufficient for optimality, and their solutions $(x^\star, \lambda^\star)$ specify all minimizers and optimal dual variables

Constrained Optimization

- **linearly-constrained optimization:**

$$\begin{array}{ll} \text{minimize}_{x \in \mathbb{R}^n} & f(x) \\ \text{s.t.} & g(x) := Ax - b = \mathbf{0} \end{array} \tag{4}$$

KKT Conditions

$$(i) \text{ constraint equation: } \mathbf{0} = g(x) = Ax - b$$

$$(ii) \text{ tangency condition: } \mathbf{0} = \frac{\partial f(x)}{\partial x} + \frac{\partial g(x)}{\partial x}^{\top} \lambda$$

1. the KKT conditions are necessary conditions for optimality of $x^\star$
2. if f is convex, then the KKT conditions are sufficient for optimality, and their solutions $(x^\star, \lambda^\star)$ specify all minimizers and optimal dual variables
3. if f is strictly convex and A has full rank, then the KKT conditions admit only a single solution $(x^\star, \lambda^\star)$

Constrained Optimization

- **linearly-constrained optimization:**

$$\begin{array}{ll} \text{minimize}_{x \in \mathbb{R}^n} & f(x) \\ \text{s.t.} & g(x) := Ax - b = \mathbf{0} \end{array} \tag{5}$$

where f is cont. diff. & **convex** and $g(x) = Ax - b$ is linear-affine

$\Rightarrow$ minimizer $x^\star$ and multiplier $\lambda^\star$ from **KKT conditions:**

(i) constraint equation: $\mathbf{0} = g(x) = Ax - b$

(ii) tangency condition: $\mathbf{0} = \dfrac{\partial f(x)}{\partial x} + \dfrac{\partial g(x)}{\partial x}^\top \lambda$

Constrained Optimization

- **linearly-constrained optimization:**

$$\begin{array}{ll} \text{minimize}_{x \in \mathbb{R}^n} & f(x) \\ \text{s.t.} & g(x) := Ax - b = \mathbf{0} \end{array} \tag{5}$$

where f is cont. diff. & **convex** and $g(x) = Ax - b$ is linear-affine

$\Rightarrow$ minimizer $x^\star$ and multiplier $\lambda^\star$ from **KKT conditions:**

(i) constraint equation: $\mathbf{0} = g(x) = Ax - b$

(ii) tangency condition: $\mathbf{0} = \dfrac{\partial f(x)}{\partial x} + \dfrac{\partial g(x)}{\partial x}^\top \lambda$

- **Lagrangian:** $\mathcal{L}(x, \lambda) = f(x) + \lambda^\top g(x)$ is convex in x and concave in λ

Constrained Optimization

- **linearly-constrained optimization:**

$$\begin{array}{ll} \text{minimize}_{x\in\mathbb{R}^n} & f(x) \\ \text{s.t.} & g(x) := Ax - b = \mathbf{0} \end{array} \tag{5}$$

where f is cont. diff. & **convex** and $g(x) = Ax - b$ is linear-affine

$\Rightarrow$ minimizer $x^\star$ and multiplier $\lambda^\star$ from **KKT conditions:**

(i) constraint equation: $\mathbf{0} = g(x) = Ax - b$

(ii) tangency condition: $\mathbf{0} = \dfrac{\partial f(x)}{\partial x} + \dfrac{\partial g(x)}{\partial x}^\top \lambda$

- **Lagrangian:** $\mathcal{L}(x, \lambda) = f(x) + \lambda^\top g(x)$ is convex in x and concave in λ

$\Rightarrow$ $\mathcal{L}(x, \lambda)$ has **saddle points**: $\mathcal{L}(x^\star, \lambda) \le \mathcal{L}(x^\star, \lambda^\star) \le \mathcal{L}(x, \lambda^\star)$

$b \in \mathbb{R}^m$ $A \in \mathbb{R}^{m\times n}$

$g(x) \in \mathbb{R}^m$:

each scalar entry of g has its own scalar λ_i associated to it

Constrained Optimization

- **linearly-constrained optimization:**

$$\begin{array}{ll} \text{minimize}_{x\in\mathbb{R}^n} & f(x) \\ \text{s.t.} & g(x) := Ax - b = \mathbf{0} \end{array} \tag{5}$$

where f is cont. diff. & **convex** and $g(x) = Ax - b$ is linear-affine

$\Rightarrow$ minimizer $x^\star$ and multiplier $\lambda^\star$ from **KKT conditions:**

(i) constraint equation: $\mathbf{0} = g(x) = Ax - b$

(ii) tangency condition: $\mathbf{0} = \dfrac{\partial f(x)}{\partial x} + \dfrac{\partial g(x)}{\partial x}^\top \lambda$

- **Lagrangian:** $\mathcal{L}(x, \lambda) = f(x) + \lambda^\top g(x)$ is convex in x and concave in λ

$\Rightarrow$ $\mathcal{L}(x, \lambda)$ has **saddle points**: $\mathcal{L}(x^\star, \lambda) \le \mathcal{L}(x^\star, \lambda^\star) \le \mathcal{L}(x, \lambda^\star)$

$\Rightarrow$ saddle points $=$ { minimizer $x^\star$ and multiplier $\lambda^\star$ } from KKT:

Constrained Optimization

- **linearly-constrained optimization:**

$$\begin{array}{ll} \text{minimize}_{x\in\mathbb{R}^n} & f(x) \\ \text{s.t.} & g(x) := Ax - b = \mathbf{0} \end{array} \tag{5}$$

where f is cont. diff. & **convex** and $g(x) = Ax - b$ is linear-affine

$\Rightarrow$ minimizer $x^\star$ and multiplier $\lambda^\star$ from **KKT conditions:**

(i) constraint equation: $\mathbf{0} = g(x) = Ax - b$

(ii) tangency condition: $\mathbf{0} = \dfrac{\partial f(x)}{\partial x} + \dfrac{\partial g(x)}{\partial x}^\top \lambda$

- **Lagrangian:** $\mathcal{L}(x, \lambda) = f(x) + \lambda^\top g(x)$ is convex in x and concave in λ

$\Rightarrow$ $\mathcal{L}(x, \lambda)$ has **saddle points**: $\mathcal{L}(x^\star, \lambda) \le \mathcal{L}(x^\star, \lambda^\star) \le \mathcal{L}(x, \lambda^\star)$

$\Rightarrow$ saddle points $=$ { minimizer $x^\star$ and multiplier $\lambda^\star$ } from KKT:

$$\mathbf{0} = \frac{\partial \mathcal{L}(x,\lambda)}{\partial x} = \frac{\partial f(x)}{\partial x} + \frac{\partial g(x)}{\partial x}^\top \lambda \qquad \mathbf{0} = \frac{\partial \mathcal{L}(x,\lambda)}{\partial \lambda} = g(x)$$

Linear Quadratic Case

$$\min \quad \tfrac{1}{2}(x-\bar{x})^T P(x-\bar{x}) := f(x)$$
$$\text{st} \quad Ax - b = 0_m \rightarrow 0 := g(x)$$

Assumptions:

1) $A \in \mathbb{R}^{m \times n}$, no assumptions yet

2) P positive semidefinite

scalar: transpose = to itself

$x^T P x \geq 0, \quad \forall\, x \in \mathbb{R}^n$

properties: $P = P^T$

P, Q are PSD $\Rightarrow P+Q$ PSD

$x^T P x$ is convex $\forall x \in \mathbb{R}^n$

$$f(x) = \tfrac{1}{2}(x-\bar{x})^T P(x-\bar{x})$$
$$= \tfrac{1}{2}x^T P x - \tfrac{1}{2}\bar{x}^T P(x-\bar{x}) - \tfrac{1}{2}x^T P \bar{x}$$
$$= \tfrac{1}{2}x^T P x - \tfrac{1}{2}\bar{x}^T P x - \tfrac{1}{2}\bar{x}^T P x + \tfrac{1}{2}\bar{x}^T P \bar{x} \qquad -\tfrac{1}{2}\bar{x}^T P^T x = -\tfrac{1}{2}\bar{x}^T P x \quad (P = P^T)$$
$$= \tfrac{1}{2}x^T P x + \tfrac{1}{2}\bar{x}^T P \bar{x} - \bar{x}^T P x$$

$$\mathcal{L}(x, \lambda) = \tfrac{1}{2}x^T P x - \bar{x}^T P x + \tfrac{1}{2}\bar{x}^T P \bar{x} + \lambda^T (Ax - b)$$

Linear Quadratic Case

$$\mathcal{L}(x,\lambda) = \frac{1}{2}x^T P x - \bar{x}^T P x + \frac{1}{2}\bar{x}^T P \bar{x} + \lambda^T (Ax - b)$$

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{1}{2}\left[Px + P^T x\right] - P^T \bar{x} + A^T\lambda$$

$$= Px - P\bar{x} + A^T\lambda = P(x - \bar{x}) + A^T\lambda = 0 \quad \text{(assume } P \text{ is PD)}$$

$$Px = P\bar{x} - A^T\lambda \Rightarrow \boxed{x^* = \underbrace{P^{-1}P}_{I}\bar{x} - P^{-1}A^T\lambda^*}$$

$$\frac{\partial \mathcal{L}}{\partial \lambda} = 0 = Ax - b = A(\bar{x} - P^{-1}A^T\lambda) - b$$

$$\Rightarrow A\bar{x} - AP^{-1}A^T\lambda - b = 0$$

$$\Rightarrow AP^{-1}A^T\lambda = A\bar{x} - b$$

(assume A is full rank, square)

"socks & shoes theorem" $(AP^{-1}A^T)^{-1} = A^{-T}PA^{-1}$

$$\lambda^* = (AP^{-1}A^T)^{-1}(A\bar{x} - b) = A^{-T}P\underbrace{A^{-1}A}_{I}\bar{x} - A^{-T}PA^{-1}b$$

$$= A^{-T}P\bar{x} - A^{-T}PA^{-1}b \hookrightarrow \text{plug this into } x^*$$

$$x^{k} = P^{-1}P\bar{x} - P^{-1}A^{T}\lambda^{k}$$

$$\lambda^{k} = A^{-T}P\bar{x} - A^{-T}PA^{-1}b$$

$$x^{k} = \bar{x} - P^{-1}A^{T}\left(A^{-T}P\bar{x} - A^{-T}PA^{-1}b\right)$$

$$= \bar{x} - P^{-1}\underbrace{A^{T}A^{-T}}_{I}P\bar{x} + P^{-1}\underbrace{A^{T}A^{-T}}_{I}PA^{-1}b$$

$$= \bar{x} - \underbrace{P^{-1}P}_{I}\bar{x} + \underbrace{P^{-1}P}_{I}A^{-1}b$$

$$= \cancel{\bar{x}} - \cancel{\bar{x}} + A^{-1}b \longrightarrow \boxed{x^{ai} = A^{-1}b}$$

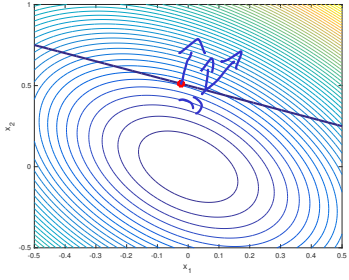
Running example for linear quadratic (LQ) case

$$\underset{x\in\mathbb{R}^n}{\text{minimize}}\ f(x)=\frac{1}{2}(x-\underline{x})^\top P(x-\underline{x})\quad g(x)=Ax-b=0$$

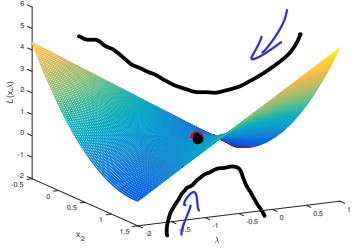
data: $P=\begin{bmatrix}2.6 & 0.8\\ 0.8 & 1.4\end{bmatrix}$, $\underline{x}=\begin{bmatrix}0\\ 0\end{bmatrix}$, $A=\begin{bmatrix}1 & 2\end{bmatrix}$, $b=1$

facts: P is positive definite with eigenvalues $\{1,3\}$, optimizer

$x^\star=[-0.0233\ 0.5116]^\top$, and optimal multiplier $\lambda^\star=-0.3488$



geometry of the LQ problem



Lagrangian projected on $x_1=0$

Primal-Dual Saddle-Point Flow

Gradient flow to unconstrained problem $\dot{x} = -k \frac{\partial f}{\partial x}$

$\mathcal{L}(x, \lambda)$ convex in x $\rightarrow$ $\dot{x} = -\frac{\partial \mathcal{L}}{\partial x} = -\frac{\partial f}{\partial x} - \frac{\partial g}{\partial x} = -\frac{\partial f}{\partial x} - A^T \lambda$

concave in λ $\rightarrow$ $\dot{\lambda} = +\frac{\partial \mathcal{L}}{\partial \lambda} = g(x) = Ax - b$

LQ example: $\mathcal{L}(x, \lambda) = \frac{1}{2}(x - \bar{x})^T P (x - \bar{x}) + \lambda^T (Ax - b)$

$\dot{x} = -\frac{\partial f}{\partial x} - A^T \lambda = -P(x - \bar{x}) - A^T \lambda$

$\dot{\lambda} = \frac{\partial \mathcal{L}}{\partial \lambda} = Ax - b$

speed it up $\begin{bmatrix} k_1^{-1} \dot{x} \\ k_2^{-1} \dot{\lambda} \end{bmatrix}$

$$\begin{bmatrix} \dot{x} \\ \dot{\lambda} \end{bmatrix} = \begin{bmatrix} -P & -A^T \\ A & 0 \end{bmatrix} \begin{bmatrix} x \\ \lambda \end{bmatrix} - \begin{bmatrix} -P\bar{x} \\ b \end{bmatrix}$$

constant input

Primal-Dual Saddle-Point Flow

Useful Matrix Lemma

Lemma: Consider a positive semidefinite matrix $P \in \mathbb{R}^{n \times n}$ and a matrix $A \in \mathbb{R}^{m \times n}$ with $n \geq m$ forming the composite **saddle matrix**

$$\mathcal{A} = \begin{bmatrix} -P & -A^\top \\ A & \mathbf{0}_{m \times n} \end{bmatrix}.$$

The matrix $\mathcal{A}$ has the following properties:

1) all eigenvalues are in the closed left half-plane: $\text{spec}(\mathcal{A}) = \{\lambda \in \mathbb{C} \mid \mathcal{R}(\lambda) \leq 0\}$. Moreover, all eigenvalues on the imaginary axis have equal algebraic and geometric multiplicity;

Useful Matrix Lemma

Lemma: Consider a positive semidefinite matrix $P \in \mathbb{R}^{n \times n}$ and a matrix $A \in \mathbb{R}^{m \times n}$ with $n \geq m$ forming the composite **saddle matrix**

$$\mathcal{A} = \begin{bmatrix} -P & -A^\top \\ A & \mathbf{0}_{m \times n} \end{bmatrix}.$$

The matrix $\mathcal{A}$ has the following properties:

1) all eigenvalues are in the closed left half-plane: $\text{spec}(\mathcal{A}) = \{\lambda \in \mathbb{C} \mid \mathcal{R}(\lambda) \leq 0\}$. Moreover, all eigenvalues on the imaginary axis have equal algebraic and geometric multiplicity;
2) if $\ker(P) \cap \text{image}(A^\top) \subseteq \{\mathbf{0}_n\}$, then $\mathcal{A}$ has no eigenvalues on the imaginary axis except for 0; and

Useful Matrix Lemma

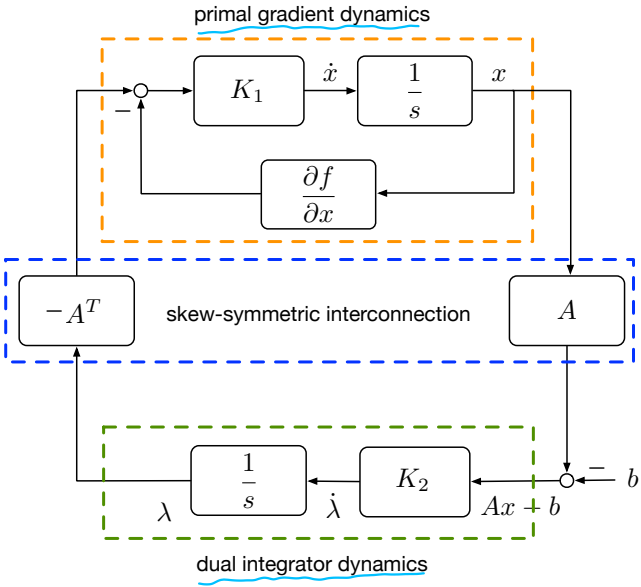
Lemma: Consider a positive semidefinite matrix $P \in \mathbb{R}^{n \times n}$ and a matrix $A \in \mathbb{R}^{m \times n}$ with $n \geq m$ forming the composite **saddle matrix**

$$\mathcal{A} = \begin{bmatrix} -P & -A^\top \\ A & \mathbf{0}_{m \times n} \end{bmatrix}.$$

The matrix $\mathcal{A}$ has the following properties:

1) all eigenvalues are in the closed left half-plane: $\text{spec}(\mathcal{A}) = \{\lambda \in \mathbb{C} \mid \mathcal{R}(\lambda) \leq 0\}$. Moreover, all eigenvalues on the imaginary axis have equal algebraic and geometric multiplicity;
2) if $\ker(P) \cap \text{image}(A^\top) \subseteq \{\mathbf{0}_n\}$, then $\mathcal{A}$ has no eigenvalues on the imaginary axis except for 0; and
3) if P is positive definite and A has full rank, then $\mathcal{A}$ has no eigenvalues on the imaginary axis.

Block-diagram of primal-dual saddle-point flow

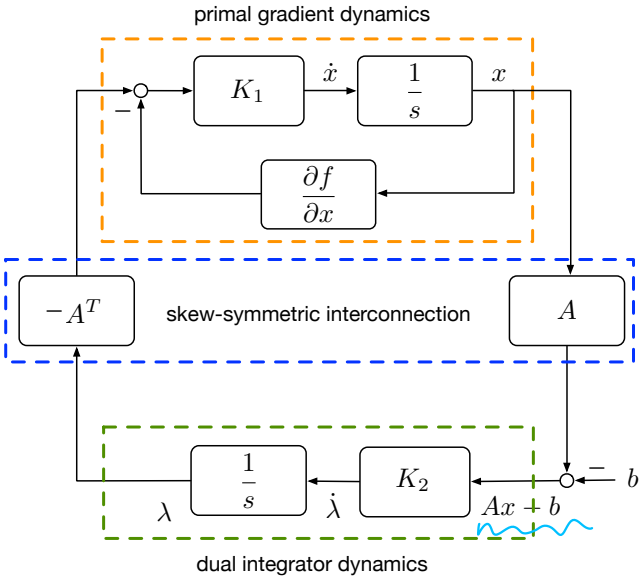


$$\dot{x} = -K_1 \frac{\partial f(x)}{\partial x} - K_1 A^\top \lambda$$

$$\dot{\lambda} = K_2 (Ax - b)$$

1. primal gradient descent

Block-diagram of primal-dual saddle-point flow

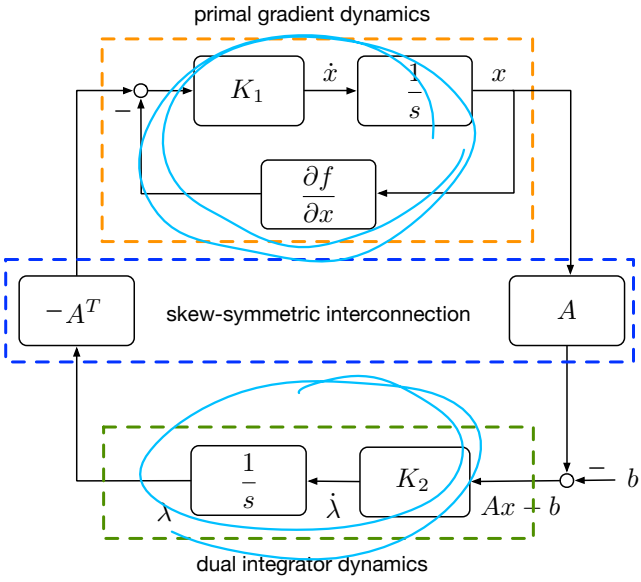


$$\dot{x} = -K_1 \frac{\partial f(x)}{\partial x} - K_1 A^\top \lambda$$

$$\dot{\lambda} = K_2 (Ax - b)$$

1. primal gradient descent
2. dual integral control penalizing constraint violation

Block-diagram of primal-dual saddle-point flow



$$\dot{x} = -K_1 \frac{\partial f(x)}{\partial x} - K_1 A^\top \lambda$$

$$\dot{\lambda} = K_2 (Ax - b)$$

1. primal gradient descent
2. dual integral control penalizing constraint violation
3. skew-symmetric interconnection

Augmented Primal-Dual Saddle-Point Flow

$$\mathcal{L}_a(x,\lambda) = f(x) + \lambda^T g(x) + \frac{\rho}{2}\|g(x)\|_2^2$$

$$= f(x) + \lambda^T (Ax-b) + \frac{\rho}{2}(Ax-b)^T(Ax-b)$$

$$\dot{x} = -k_1 \frac{\partial \mathcal{L}_a}{\partial x} = -k_1 \frac{\partial f}{\partial x} - k_1 A^T\lambda - \rho k_1 A^T(Ax-b)$$

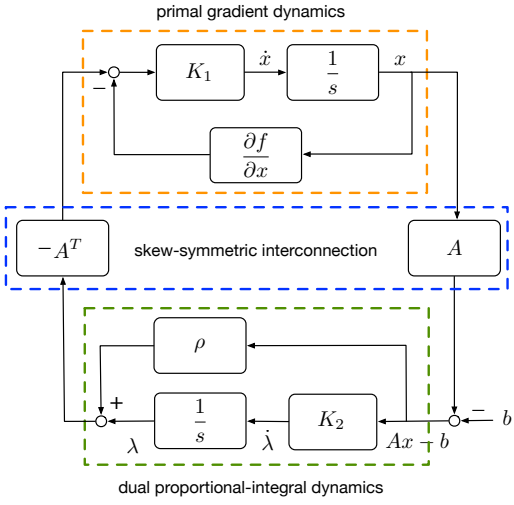
$$\dot{\lambda} = k_2 (Ax-b)$$

2-norm

$\|x\|_2^2$ is $x^Tx = \sum_{i=1}^{n} x_i^2$

proportional control on constraint violation

Block-diagram of augmented primal-dual saddle-point flow

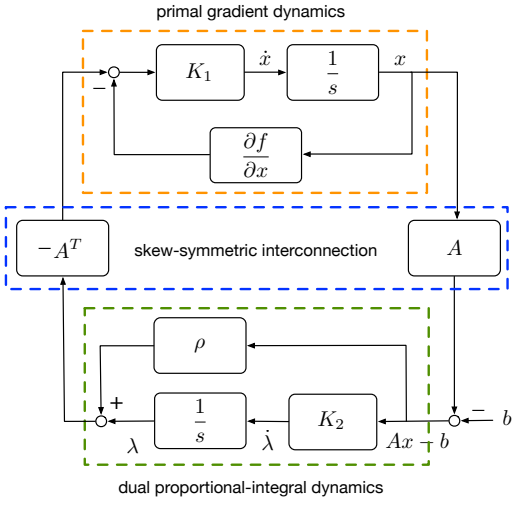


$$\dot{x} = -K_1 \frac{\partial f(x)}{\partial x} - K_1 A^\top \lambda$$
$$-\rho K_1 A^T (Ax - b)$$

$$\dot{\lambda} = K_2 (Ax - b)$$

1. primal gradient descent

Block-diagram of augmented primal-dual saddle-point flow

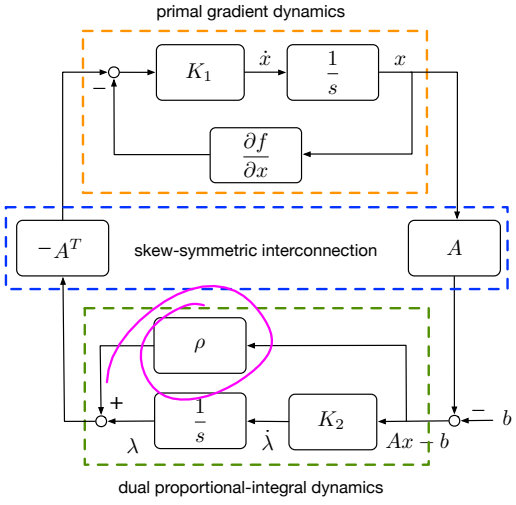


$$\dot{x} = -K_1 \frac{\partial f(x)}{\partial x} - K_1 A^\top \lambda$$
$$-\rho K_1 A^T (Ax - b)$$

$$\dot{\lambda} = K_2 (Ax - b)$$

1. primal gradient descent
2. dual proportional-integral control penalizing constraint violation

Block-diagram of augmented primal-dual saddle-point flow



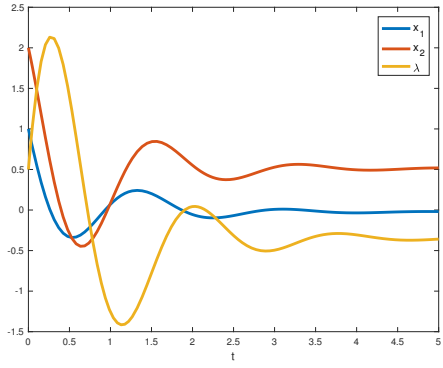
$$\dot{x} = -K_1 \frac{\partial f(x)}{\partial x} - K_1 A^\top \lambda$$
$$-\rho K_1 A^T (Ax - b)$$

$$\dot{\lambda} = K_2 (Ax - b)$$

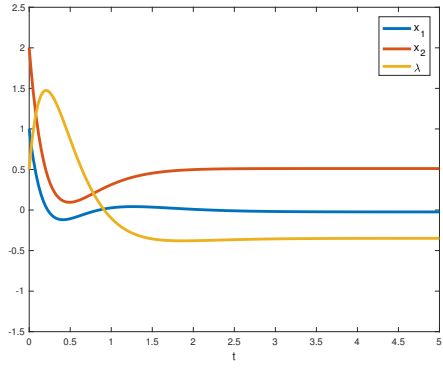
1. primal gradient descent
2. dual proportional-integral control penalizing constraint violation
3. skew-symmetric interconnection

Linear-Quadratic Case

Standard and augmented saddle-point flow for LQ program



standard saddle-point flow



augmented saddle-point flow ($\rho = 1$)

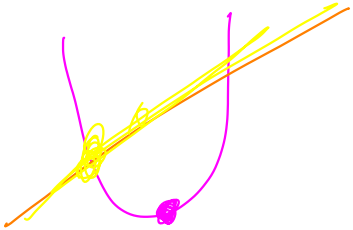
augmentation induces (stricter) convexity & better performance (damping)

Punchline + Caveat

- Algorithms (or rather, dynamics) find the optimal point

Punchline + Caveat

- Algorithms (or rather, dynamics) find the optimal point
- Caveat: nobody does this in continuous time...

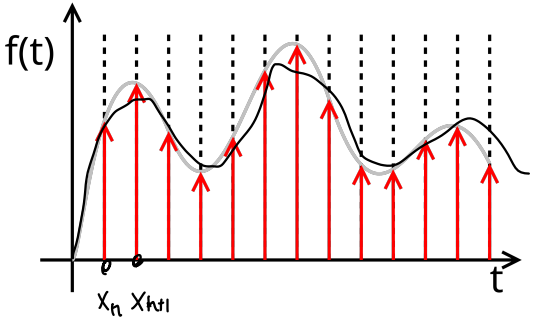


Optimal Control

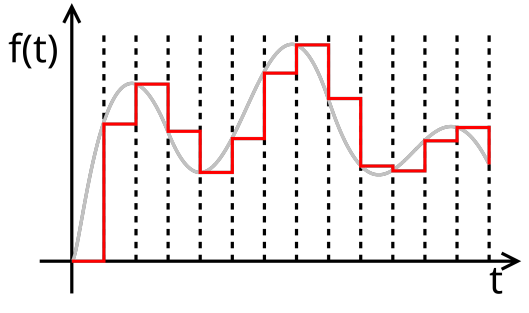
Discrete-Time Systems

$\dot{x} = f(x) \longrightarrow x_{n+1} = f(x_n)$

This stuff is a bit more intuitive in **discrete time**, so let's introduce that here.



Sampling



Sample-and-hold

Discrete-Time Systems

$$\dot{x} = Ax + Bu$$

$$\downarrow$$

$$\hat{x}_{k+1} = \bar{A}\hat{x}_k + \bar{B}\hat{u}_k$$
$$\hat{y}_k = \bar{C}\hat{x}_k + \bar{D}\hat{u}_k$$

where the discrete-time matrices are (for a timestep Δt):

$$\bar{A} = e^{A\Delta t}, \ \bar{B} = A^{-1}\left(e^{A\Delta t} - I\right)B$$
$$\bar{C} = C, \ \bar{D} = D.$$

Optimal Control

Static Optimization

$$\begin{array}{ll} \min\limits_{x \in \mathbb{R}^n} & \frac{1}{2} x^T P x \\ \text{s.t.} & Ax = b \end{array}$$

Dynamic Optimization

$$\begin{array}{ll} \min\limits_{x \in \mathbb{R}^n \times \mathcal{T}} & \frac{1}{2} \int_{\mathcal{T}} \left[x^T P x + u^T R u \right] dt \\ \text{s.t.} & \dot{x} = Ax + Bu \end{array}$$

Optimal **points** versus optimal **trajectories**

Optimal Control

Static Optimization

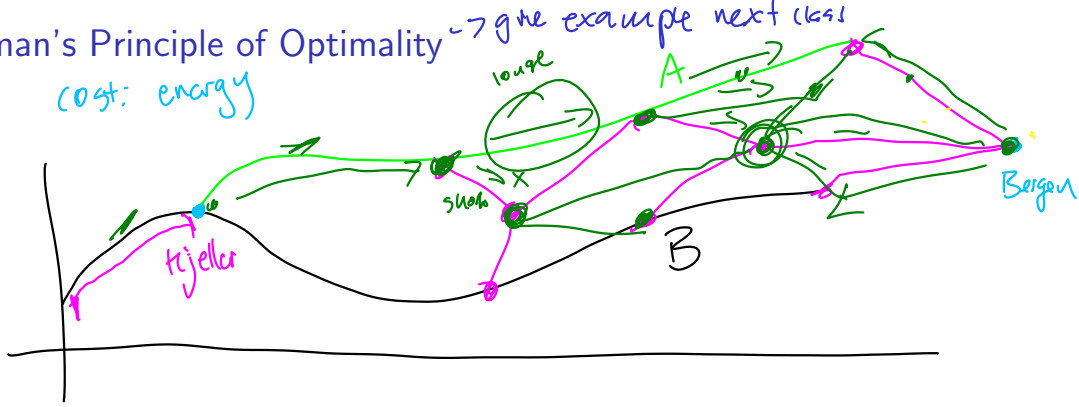
$$\begin{array}{ll} \underset{x \in \mathbb{R}^n}{min} & \frac{1}{2} x^T P x \\ \text{s.t.} & Ax = b \end{array}$$

Dynamic Optimization

$$\begin{array}{ll} \underset{x \in \mathbb{R}^n \times [0,T]}{\min} & \frac{1}{2} \sum_{k=1}^{T} \left[x_k^T P_k x_k + u_k^T R_k u_k \right] \\ \text{s.t.} & x_{k+1} = \bar{A} x_k + \bar{B} u_k \end{array}$$

Optimal **points** versus optimal **sequence**

Bellman's Principle of Optimality



- The optimal trajectory from where you are doesn't depend on where you've been
- Eg., sunken cost fallacy
- Dynamic programming: start at the end, go back, optimize 'cost-to-go'

Discrete-Time LQR → Linear quadric regulator

$$\min_{\{x_n\},\{u_n\}} \sum_{n=1}^{T} \left[x_{n+1}^T Q x_{n+1} + u_n^T R u_n \right] \qquad Q \succ 0,\ R \succ 0$$

$x_{n+1}^T Q x_{n+1}$: cost of being away from 0; $u_n^T R u_n$: "energy" or "effort"

$$\text{s.t.} \quad x_{n+1} = A x_n + B u_n, \quad x_1 = \bar{x}$$

value function $V_t(x_t) = \min_{u_t, \dots, u_{T-1}} \sum_{k=t}^{T-1} \left(x_n^T Q x_n + u_n^T R u_n \right) + x_T^T Q x_T$

$$= \min_{\{u_t\}} \left(x_t^T Q x_t + u_t^T R u_t + \left[\sum_{n=t+1}^{T-1} \left(x_n^T Q x_n + u_n^T R u_n \right) + x_T^T Q x_T \right] \right)$$

$$\left[\sum_{n=t+1}^{T-1} \left(x_n^T Q x_n + u_n^T R u_n \right) + x_T^T Q x_T \right] = V_{t+1}(x_{t+1})$$

→ recursion

$$V_t(x) = \min_u \left(x^T Q x + u^T R u + V_{t+1}(x_{t+1}) \right)$$

$$= \min_u \left(x^T Q x + u^T R u + V_{t+1}(Ax + Bu) \right)$$

Discrete-Time LQR

Ansatz: guess the solution & plug it in

Suppose $V_{t+1}(x) = x^T S_{t+1} x$, $S_{t+1} = S_{t+1}^T \succ 0$

$$V_t(x) = x^T Q x + \min_u \left(u^T R u + (Ax+Bu)^T S_{t+1} (Ax+Bu) \right)$$

Quadratic problem, did this derivative already!

$$\frac{d}{du}(\cdot) = 0 = 2u^T R + 2(Ax+Bu)^T S_{t+1} B = 0 \quad \text{solve for } u^*$$

$$= 2u^T R + 2u^T B^T S_{t+1} B + 2x^T A^T S_{t+1} B = 0$$

$$= u^T (R + B^T S_{t+1} B) + x^T A^T S_{t+1} B = 0 \quad \text{(transpose both sides)}$$

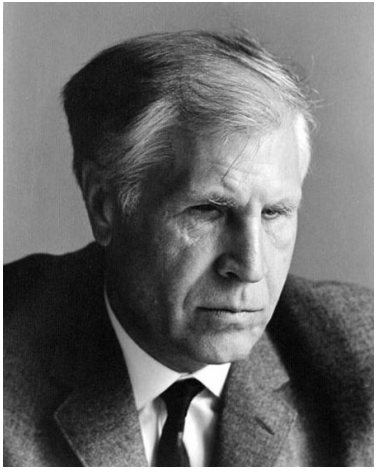
$$u^* = -(R + B^T S_{t+1} B)^{-1} B^T S_{t+1} A x$$

↳ Next time, derive S_{t+1}

Discrete-Time LQR

Pontryagin Maximum Principle

What about the **KKT conditions** for **dynamic problems**?



Super general problem:

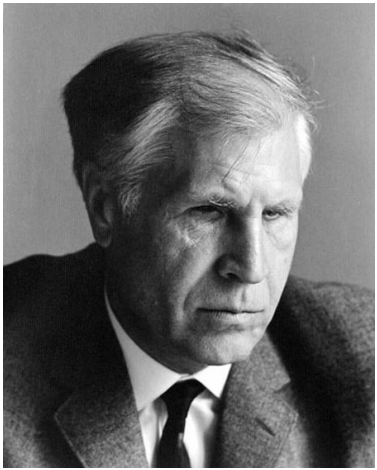
$$\begin{array}{ll} \min & \phi(x(T), T) + \int_0^T L(x, u, t)dt \\ \text{s.t.} & \psi(x(T), T) = 0 \\ & \dot{x} = f(x, u, t) \end{array}$$

Define the **Hamiltonian:**

$$H(x, u, \lambda, t) := L(x, u, \lambda, t) + \lambda(t)^T f(x, u, t)$$

Pontryagin Maximum Principle

What about the **KKT conditions** for **dynamic problems**?



$$H(x, u, \lambda, t) := L(x, u, \lambda, t) + \lambda(t)^T f(x, u, t)$$

Pontryagin's maximum principle:

$$H(x^*, u^*, \lambda^*, t) \leq H(x^*, u, \lambda^*, t), \ \forall t \in [0, T]$$

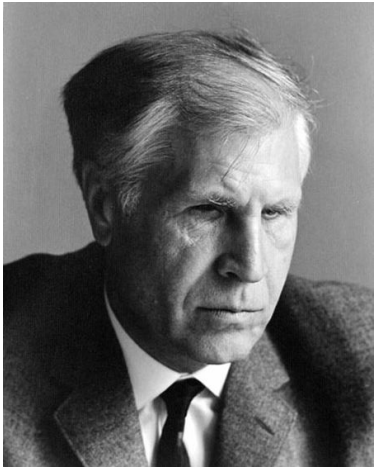
Coupled DEs:

$$\dot{x} = \frac{\partial H}{\partial \lambda} = f$$

$$-\dot{\lambda} = \frac{\partial H}{\partial x} = \frac{\partial f}{\partial x}^T \lambda + \frac{\partial L}{\partial x}$$

Pontryagin Maximum Principle

What about the **KKT conditions** for **dynamic problems**?



Coupled DEs:

$$\dot{x} = \frac{\partial H}{\partial \lambda} = f$$

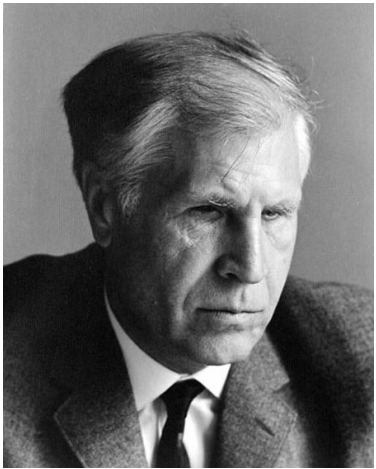
$$-\dot{\lambda} = \frac{\partial H}{\partial x} = \frac{\partial f}{\partial x}^T \lambda + \frac{\partial L}{\partial x}$$

Boundary conditions:

$$0 = \frac{\partial H}{\partial u} = \frac{\partial L}{\partial u} + \frac{\partial f}{\partial u}^T \lambda$$

Pontryagin Maximum Principle

What about the **KKT conditions** for **dynamic problems**?



Coupled DEs:

$$\dot{x} = \frac{\partial H}{\partial \lambda} = f$$

$$-\dot{\lambda} = \frac{\partial H}{\partial x} = \frac{\partial f}{\partial x}^T \lambda + \frac{\partial L}{\partial x}$$

Boundary conditions:

$$\left(\phi_x + \psi_x^T v - \lambda\right)^T \Big|_T dx(T) + \left(\phi_t + \psi_t^T v + H\right) \Big|_T dT = 0$$

Continuous-Time LQR vs Discrete-Time LQR

Discrete-Time LQR

$$\begin{aligned} \min_{x \in \mathbb{R}^n \times [0,T]} \quad & \frac{1}{2} \sum_{k=1}^{T} \left[x_k^T P_k x_k + u_k^T R_k u_k \right] \\ \text{s.t.} \quad & x_{k+1} = \bar{A} x_k + \bar{B} u_k \end{aligned}$$

Solution:

$$\begin{aligned} J_k^* &= \frac{1}{2} x_k^T S_k x_k \\ S_k &= \bar{A} T S_{k+1} \bar{A} - \bar{A}^T S_{k+1} \bar{B} K_k + P_k \\ K_k &= (B^T S_{k+1} B + R_k)^{-1} B^T S_{k+1} \bar{A} \\ u_k^* &= -K_k x_k \end{aligned}$$

Continuous-Time LQR

$$\begin{aligned} \min_{x \in \mathbb{R}^n \times \mathcal{T}} \quad & \frac{1}{2} \int_{\mathcal{T}} \left[x^T P x + u^T R u \right] dt \\ \text{s.t.} \quad & \dot{x} = Ax + Bu \end{aligned}$$

Solution:

$$\begin{aligned} J^*(t) &= \frac{1}{2} x(t)^T S(t) x(t) \\ -\dot{S} &= SA + A^T S - SBR^{-1} B^T S + Q \\ K &= -R^{-1} B^T S \\ u &= -Kx \end{aligned}$$

Motivation for MPC

In practice, LQR can tune gains really nicely

Some cons:

- Cannot handle constraints
- Computationally intensive (finite time-horizon)
- No guaranteed margins when noise in system and estimation loops

Guaranteed Margins for LQG Regulators

JOHN C. DOYLE

***Abstract*—There are none.**

Motivation for MPC

$$\begin{aligned} \min_{x \in \mathbb{R}^n \times [0,T]} \quad & \frac{1}{2} \sum_{k=1}^{T} \left[x_k^T P_k x_k + u_k^T R_k u_k \right] \\ \text{s.t.} \quad & x_{k+1} = \bar{A} x_k + \bar{B} u_k \\ & x_k \in \mathcal{X}_k \\ & u_k \in \mathcal{U}_k \end{aligned}$$

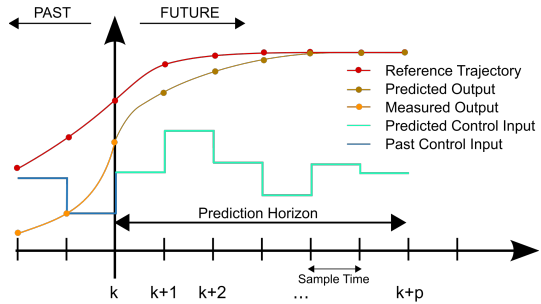
Model Predictive Control

MPC

- At each timestep k solve:

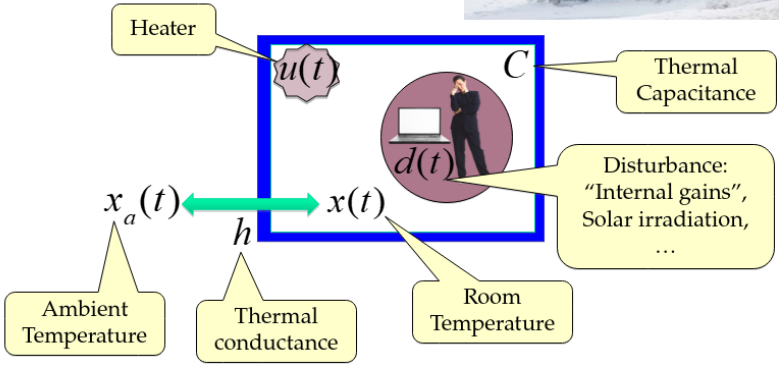
$$\begin{aligned} \min \quad & \textstyle\sum_{t=k}^{k+p} c_k(x_k, u_k) \\ \text{s.t.} \quad & x_{t+1} = f(x_t, u_t) \\ & x_k = x(k) \\ & x_t \in \mathcal{X} \\ & u_t \in \mathcal{U} \end{aligned}$$

- Apply the first control input $u(k) = u_k$
- $k \mapsto k + 1$
- Repeat



Building Control

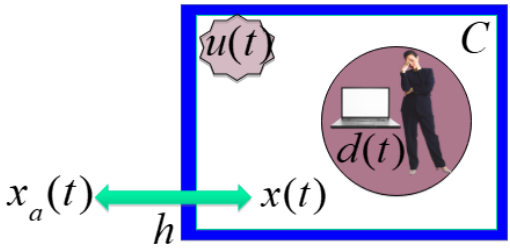
Single room house



Modeling Buildings – Single Room

$$C\frac{d}{dt}x(t) = h\left[x_a(t) - x(t)\right] + u(t) + d(t)$$

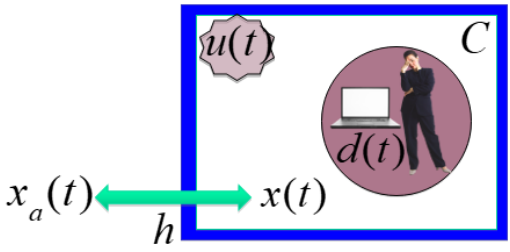
- C: Thermal capacity of the room



Modeling Buildings – Single Room

$$C\frac{d}{dt}x(t) = h\left[x_a(t) - x(t)\right] + u(t) + d(t)$$

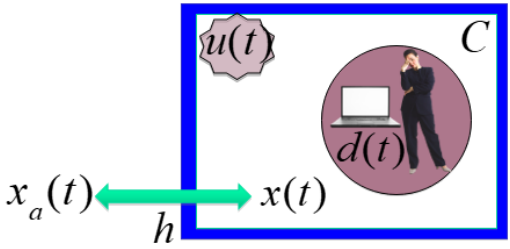
- C: Thermal capacity of the room
- h: Insulation of the wall



Modeling Buildings – Single Room

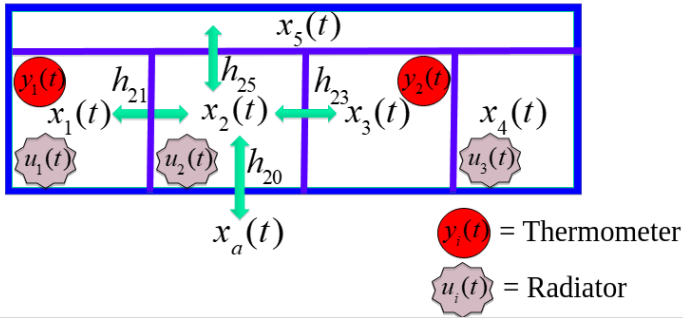
$$C\frac{d}{dt}x(t) = h\left[x_a(t) - x(t)\right] + u(t) + d(t)$$

- C: Thermal capacity of the room
- h: Insulation of the wall
- $u(t)$: Heat input at time t



Modeling Buildings – Multiple Zones

Multiple zones



$$C_2\dot{x}_2(t) = h_{20}(x_a(t)-x_2(t))+h_{21}(x_1(t)-x_2(t))+$$
$$h_{23}(x_3(t)-x_2(t))+h_{25}(x_5(t)-x_2(t))+u_2(t)+d_2(t)$$

Modeling Buildings – Multiple Zones

$$x(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \\ x_3(t) \\ x_4(t) \\ x_5(t) \end{bmatrix}, \quad u(t) = \begin{bmatrix} u_1(t) \\ u_2(t) \\ u_3(t) \end{bmatrix}, \quad d(t) = \begin{bmatrix} d_1(t) \\ d_2(t) \\ d_3(t) \\ d_4(t) \\ d_5(t) \end{bmatrix} \tag{6}$$

$$H_{ij} = \begin{cases} \frac{h_{ij}}{C_i} & \text{if } i \text{ is a neighbour of } j \\ 0 & \text{else} \end{cases} \tag{7}$$

$$H_{ii} = \sum_{j=1,\ldots,5} H_{ij} \tag{8}$$

Modeling Buildings – Multiple Zones

$$\dot{x}(t) = \begin{bmatrix} -H_{11} & H_{12} & 0 & 0 & H_{15} \\ H_{21} & -H_{22} & H_{23} & 0 & H_{25} \\ 0 & H_{32} & H_{33} & H_{34} & H_{35} \\ 0 & 0 & H_{43} & -H_{44} & H_{45} \\ H_{51} & H_{52} & H_{53} & H_{54} & -H_{55} \end{bmatrix} x(t) + \begin{bmatrix} H_{10} \\ H_{20} \\ H_{30} \\ H_{40} \\ H_{50} \end{bmatrix} x_a(t) + \tag{9}$$

$$\begin{bmatrix} \frac{1}{C_1} & 0 & 0 \\ 0 & \frac{1}{C_2} & 0 \\ 0 & 0 & 0 \\ 0 & 0 & \frac{1}{C_4} \\ 0 & 0 & 0 \end{bmatrix} u(t) + \begin{bmatrix} \frac{1}{C_1} & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{C_2} & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{C_3} & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{C_4} & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{C_5} \end{bmatrix} d(t) \tag{10}$$

Modeling Buildings – Multiple Zones

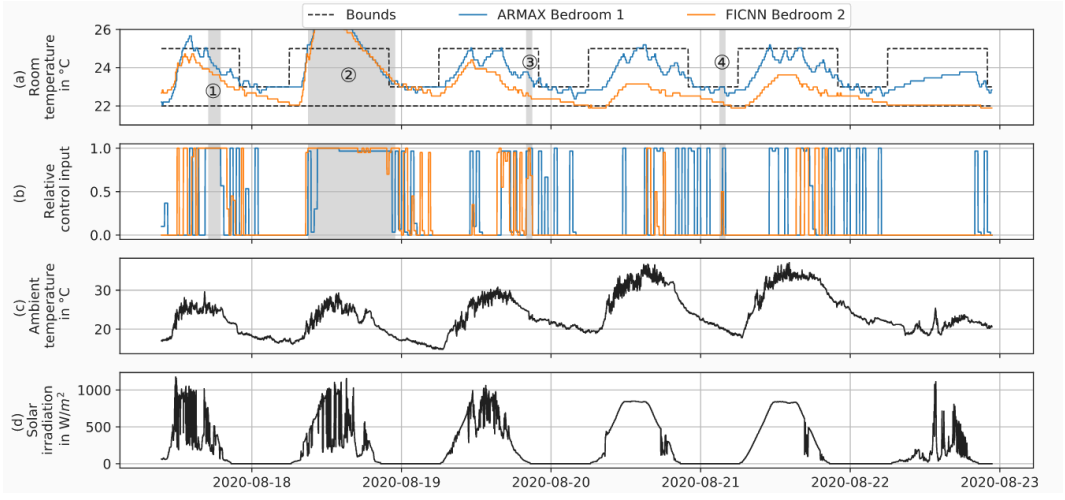
$$\dot{x} = Ax(t) + bx_a(t) + Bu(t) + Dd(t) \tag{11}$$

$$y(t) = Cx(t) + cx_a(t) \tag{12}$$

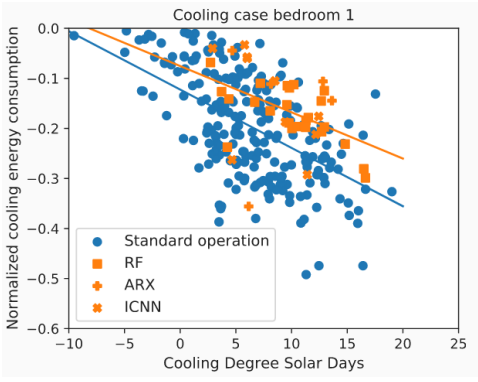
Example of Building MPC

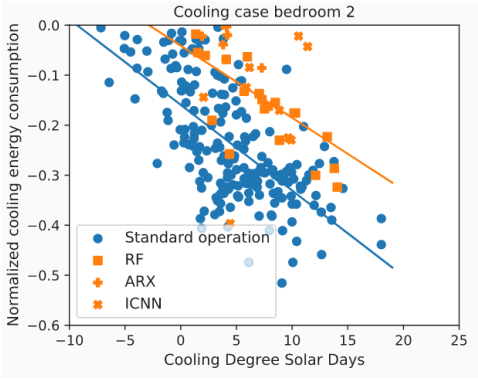
Empa
Materials Science and Technology

Performance of ICNNs In the Loop



Comparison with other ML Techniques + Hysteresis Control





Punchline: MPC consumes 33% less cooling energy at 5 CDSD and 28% less at 15 CDSD.

[0] Felix Bünning, Benjamin Huber, Adrian Schalbetter, Ahmed Aboudonia, Mathias Hudoba de Badyn, Philipp Heer, Roy S Smith, John Lygeros. *Physics-informed linear regression is competitive with two Machine Learning methods in residential building MPC.* Applied Energy 310 (2022) 118491.

Next Time

- Observability ("Where am I?" from sensors)

Next Time

- Observability ("Where am I?" from sensors)
- Kalman Filter (more black magic)

Next Time

- Observability ("Where am I?" from sensors)
- Kalman Filter (more black magic)
- Spacecraft GNC